MOBILE WEATHER STATION.

github project repository

1. INTRODUCTION

Background:

Weather monitoring is an important aspect in agriculture, environmental research, and urban planning. Access to localized weather data allows individuals and organizations to make better decisions related to farming activities, outdoor operations, and environmental monitoring. While large-scale weather stations exist, they are often expensive and limited to specific locations, making it difficult to obtain highly localized environmental data.

Advancements in embedded systems and low-power microcontrollers, have made it possible to design compact and affordable weather monitoring devices. These devices can collect environmental data such as temperature and humidity, and make this information available for analysis or real-time monitoring.

Project Overview:

This project focuses on the design and development of a compact embedded weather station capable of measuring and reporting environmental conditions in real time. The system will be built using a microcontroller-based architecture combined with environmental sensors.

The weather station will collect weather-related parameters such as temperature and humidity through dedicated sensors. The data will be processed by the embedded system firmware and presented through a web interface that can be accessed locally or remotely.

The project emphasizes a modular hardware and software architecture, allowing clear separation between the hardware abstraction layer, device drivers, and application logic.

Scope:

The scope of this project includes the design, implementation, and testing of a small-scale embedded weather station system.

The project will cover both hardware and software aspects of the system, including sensor integration, firmware development, power management, and system documentation.

The system will operate as a standalone device powered by a battery source.

The design will also include schematic diagrams, PCB layout, and enclosure considerations to demonstrate a complete embedded product.

Objectives:

The main objectives of this project are:

• To design and implement a battery-powered embedded weather station.
• To collect environmental data such as temperature and humidity using digital sensors.
• To develop modular firmware using layered architecture (HAL, drivers, and application logic).
• To design the complete hardware system including schematic diagrams and PCB layout.
• To demonstrate professional embedded system development practices including documentation, testing, and system validation.

2. PROBLEM STATEMENT:

Goal:

To design and implement a compact, battery-powered embedded weather station capable of measuring environmental conditions such as temperature and humidity and presenting the data through a web-based dashboard.

Conflict:

As professional weather stations are expensive and deployed only in selected locations. Individuals, students, and small-scale users often lack affordable tools that allow them to monitor environmental conditions in their immediate surroundings.

Additionally, many existing solutions require wired power sources or complex infrastructure, which limits their portability and practical use in remote or off-grid environments.

Impact:

The lack of accessible environmental monitoring devices prevents users from collecting useful environmental data that could assist in activities such as, small-scale agriculture, environmental observation, or educational experimentation.

Proposed Solution:

This project proposes the development of a battery-powered embedded weather station using an ESP32 microcontroller, integrated with temperature and humidity sensors to collect environmental data at regular intervals. The collected data will be processed by the embedded firmware and served through a web-based dashboard hosted by the ESP32.

Users will be able to access the dashboard through a web browser connected to the same network, allowing real-time monitoring of environmental conditions.

Phase 2: System Requirements

1. FUNCTIONAL REQUIREMENTS.

Environmental Data Acquisition

The system shall measure ambient temperature and relative humidity using the DHT22 sensor.

1. Periodic Data Sampling
   The ESP32 firmware shall periodically read temperature and humidity data from the sensor at defined intervals.
2. Data Processing
   The microcontroller shall process the sensor readings and convert them into human-readable values.
3. Web Dashboard Interface
   The system shall host a web-based dashboard that displays the measured temperature and humidity values.
4. Wireless Connectivity
   The ESP32 shall operate in Access Point mode, allowing nearby devices to connect directly to the weather station through Wi-Fi.
5. Client Access
   Users shall be able to access the dashboard using a web browser by connecting to the ESP32 network.
6. Real-Time Data Display
   The web dashboard shall display the latest temperature and humidity readings in real time or near real time.
7. System Initialization
   The system shall automatically start the sensor and web server services when the device powers on.
8. Error Handling
   The firmware shall detect and handle sensor read errors and notify the system when invalid readings occur.
9. System Reset Capability
   The system shall allow firmware reset through the ESP32 reset mechanism.
10. System Reset Capability

    The system shall allow firmware reset through the ESP32 reset mechanism.

2. NON FUNCTIONAL REQUIREMENTS

1. System Reliability
   The system shall operate continuously without crashing or requiring frequent resets.
2. Low Power Operation
   The weather station shall operate using a rechargeable 18650 Li-ion battery and must be designed to minimize unnecessary power consumption.
3. Modularity
   The firmware shall follow a layered architecture separating HAL, drivers, and application services to improve maintainability and scalability.
4. Maintainability
   The codebase shall be structured in modular components to allow future upgrades such as adding more sensors.
5. Scalability
   The system architecture shall allow integration of additional environmental sensors in the future.
6. Usability
   The web dashboard shall present environmental data in a clear and simple interface accessible through standard web browsers.
7. Portability
   The software design shall avoid direct hardware dependencies in the application layer by using abstraction layers.
8. System Startup Time
   The system shall initialize and begin sensor data collection within a reasonable time after power-up.

Phase 3: System Architecture.

1. SYSTEM BLOCK DIAGRAM

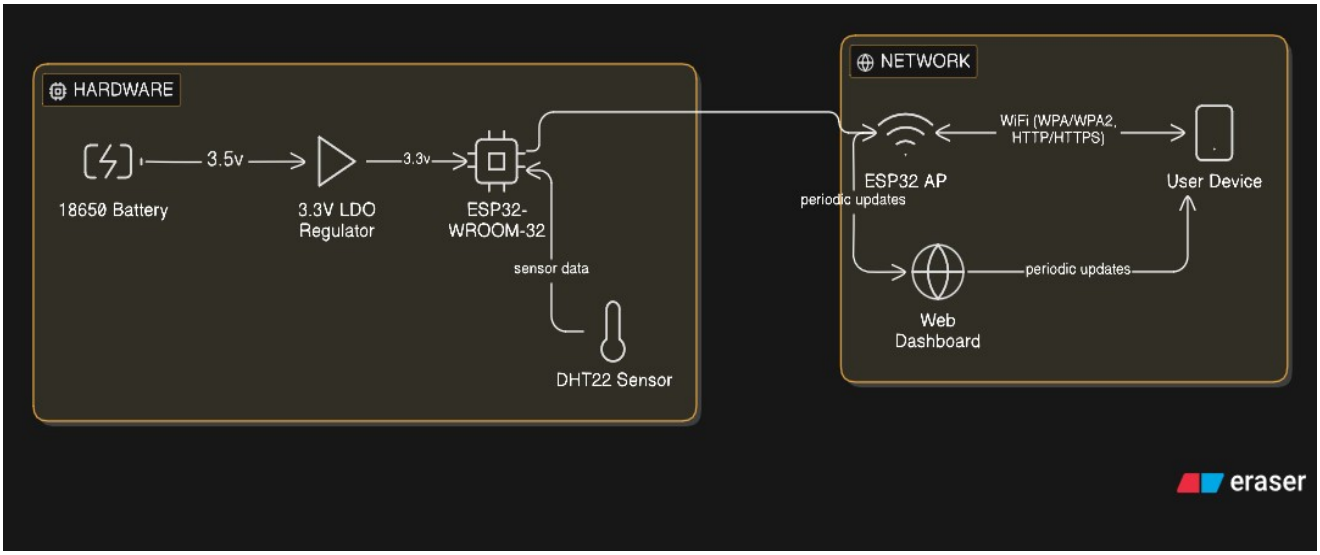


Figure 1: The Block diagram for the circuitry.

2. HARDWARE DESIGN.

Microcontroller Unit.

The system is built around the **ESP32-WROOM-32** microcontroller module, due to its integrated Wi-Fi capability, and support for embedded web server applications.

Sensor Interface.

Temperature and humidity measurements are obtained using the **DHT22 sensor**. The sensor communicates with the microcontroller using a single-wire digital communication protocol.

Power Supply System.

The weather station is powered using a single **18650 Li-ion** rechargeable battery. The battery provides a nominal voltage of approximately 3.7V.

Voltage Regulation.

Since the ESP32 and DHT22 operate at 3.3V, a Low Dropout (LDO) voltage regulator is used to regulate the battery voltage to a stable 3.3V output.

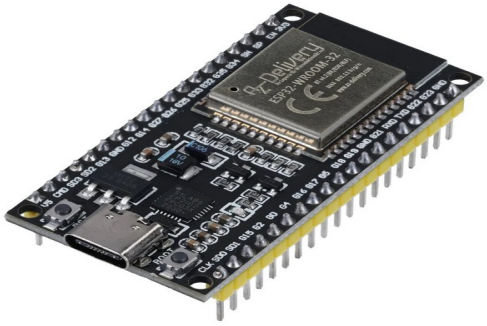
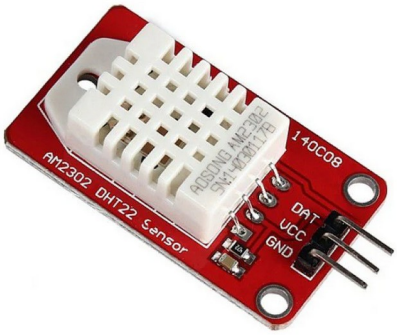
Battery Charging.

The system uses an external battery charger for charging the 18650 cell. This simplifies the internal circuit design and allows safe charging without adding additional charging circuitry inside the device.

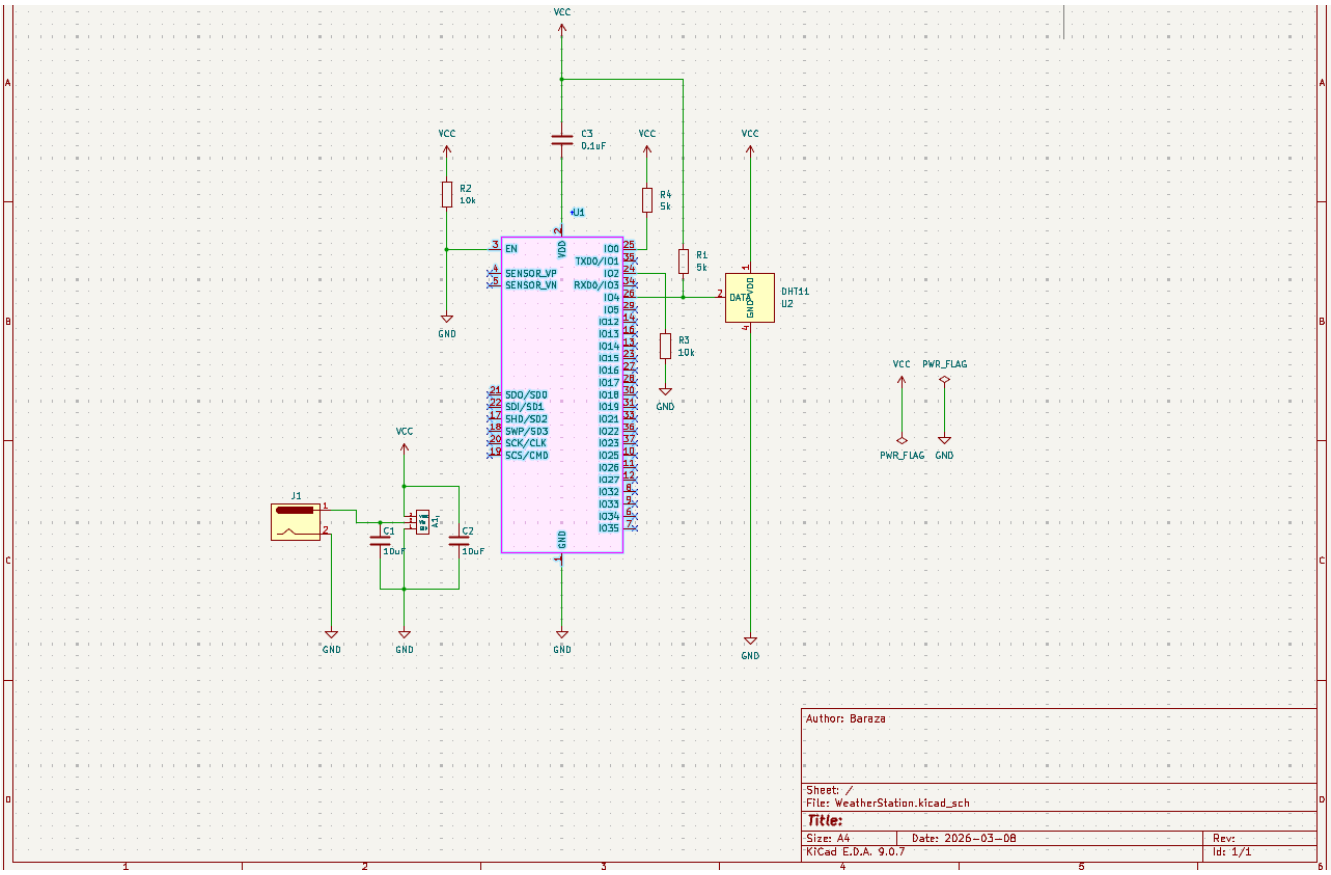
Wireless Communication.

The ESP32 operates in **Access Point mode**, creating its own **Wi-Fi** network. Users can connect directly to the device using a smartphone or computer and access the web dashboard through a browser.

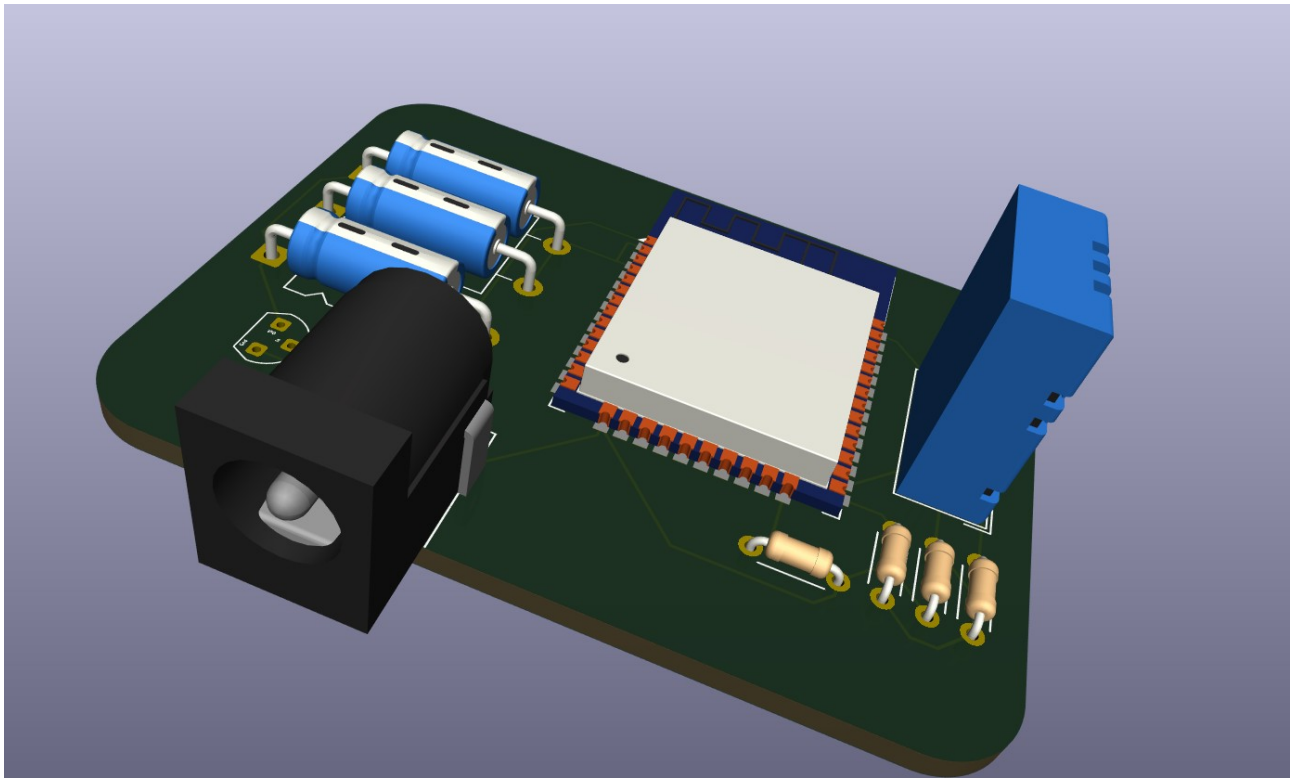
COMPONENT SELECTION JUSTIFICATION.

Item #	Device Name.	Qty	Image Reference	Cost.
1	ESP32-WROOM-32	1		28,000/=
2	DHT22 (AM2302)	1		18,000/=
3	18650 Li-ion Cell	1		5,000/=
4	HT7333-A (LDO)	1		1,000/=

THE SCHEMATIC DIAGRAM OF THE CIRCUIT.



THE 3D VIEW OF THE CIRCUITRY.



3. SOFTWARE ARCHITECTURE.

The firmware follows a modular **layered architecture designed** to improve maintainability, scalability, and portability of the embedded software.

The architecture consists of the following layers:

1. Application Layer.

This is the **main** system logic and coordinates the operation of the entire weather station. The application layer **initializes system components** during startup and manages the execution of different services.

2. Services Layer.

The services layer contains **higher-level system functionalities** that combine multiple drivers to provide meaningful services to the application layer. In this system, services manage weather data processing and the web interface.

Examples of services in the system include:

Weather Service – Responsible for collecting temperature and humidity data from the sensor driver and storing the latest readings.

Web Service – Responsible for running the embedded web server and providing the dashboard interface to connected users.

3. Drivers Layer.

The drivers layer provides **device-specific functionality** for external hardware components. Drivers interact with the hardware abstraction layer (**HAL**) rather than directly to a microcontroller.

- This improves portability and allows the firmware to be adapted to different hardware platforms.

For this project, the primary driver is:

DHT22 Driver - Responsible for communicating with the DHT22 sensor and retrieving temperature and humidity data.

4. Hardware Abstraction Layer (**HAL**).

HAL isolates the firmware from microcontroller-specific implementations.

This layer **communicates directly** with **ESP32** while exposing hardware-independent interfaces to the upper layers of the firmware.

5. Hardware Layer.

Represents the **physical hardware components** used in the system, including the ESP32 microcontroller, the DHT22 sensor, the voltage regulator, and the battery power supply.

6. System Tasks (FreeRTOS)

The system firmware uses FreeRTOS to **run concurrent tasks** that handle different operations of the weather station.

Sensor Task.

Responsible for periodically reading temperature and humidity values from the DHT22 sensor. The task runs at fixed intervals of **30 seconds** and updates the latest environmental data.

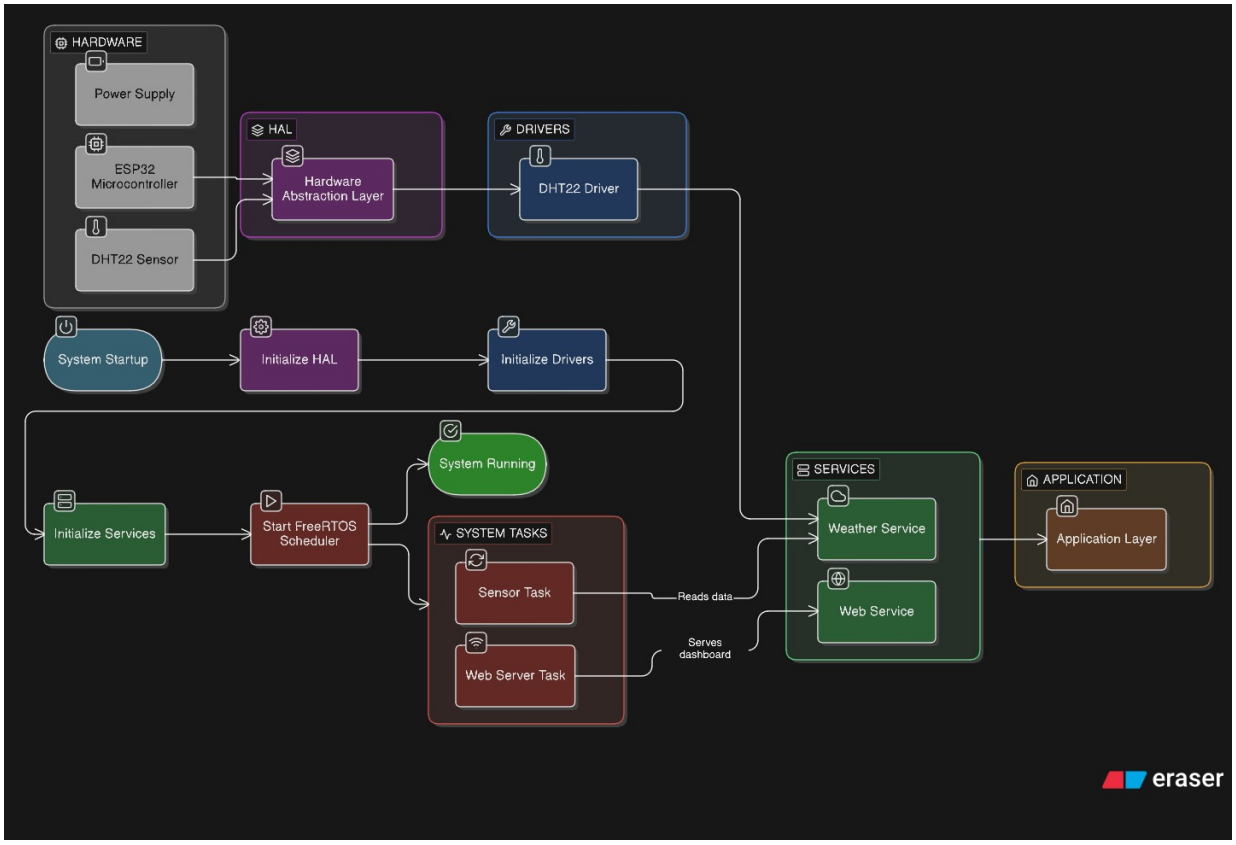
Web Server Task.

Runs the embedded HTTP server that provides the web dashboard. It listens for incoming HTTP requests from connected clients and responds with the latest sensor readings.

System Initialization

During system startup, the firmware initializes the HAL layer, sensor drivers, and services before starting the FreeRTOS scheduler. Once initialized, the system begins executing the sensor task and the web server task **concurrently**.

THE SYSTEM OVERALL FLOWCHART.



Phase 4: Implementation and Testing.

A. Test Plan.

1. Hardware Validation.

Before running the firmware, the hardware components of the weather station will be verified to ensure proper electrical operation.

Power System Test.

The output of the 3.3V LDO regulator will be measured using a multimeter to confirm that it provides a stable voltage suitable for the ESP32 and the DHT22 sensor.

Microcontroller Power Test.

The **ESP32-WROOM-32** module will be powered through the regulated 3.3V supply to confirm proper boot behavior.

Sensor Connection Test.

The **DHT22** sensor connections will be verified to ensure correct wiring of VCC, GND, and the data pin connected to the ESP32 GPIO.

Battery Integration Test.

The **18650 Li-ion battery** will be connected to the power regulation circuit to verify that the system powers on correctly when operating from battery power.

2. Software Validation.

Sensor Communication Test.

The firmware will be tested to verify that the ESP32 can successfully read temperature and humidity data from the **DHT22** sensor.

Web Server Test

The **ESP32** will be configured in Access Point (**AP**) mode, and a client device such as a smartphone or laptop will connect to the network. The web dashboard will be accessed through a browser to verify that sensor data is displayed correctly.

Data Update Test.

The system will be observed to confirm that sensor readings are updated every 30 seconds as defined in the system requirements.

Deep Sleep Functionality Test.

The firmware will be tested to confirm that the **ESP32** enters deep sleep mode after completing sensor readings and wakes up periodically to collect new data.

System Stability Test.

The weather station will be run continuously for extended periods to verify reliable operation without firmware crashes or system resets.

Results and Analysis.

After implementation, the weather station will be evaluated to determine how well it meets the system requirements.

Performance Evaluation.

The accuracy and consistency of the temperature and humidity readings will be analyzed based on the data provided by the DHT22 sensor.

Power Consumption.

The use of ESP32 deep sleep mode evaluated to determine its effectiveness in reducing power consumption and extending battery life.

System Responsiveness.

Web dashboard evaluated to ensure that users can access environmental data quickly and reliably through the ESP32 Access Point network.

Reliability.

The system tested under continuous operation to confirm that it remains stable without unexpected failures.

Phase 5: Support and Maintenance.

USER GUIDE

System Setup

1. Insert a fully charged **18650 Li-ion battery** into the weather station battery holder.
2. Power on the device. Once powered, the **ESP32** will automatically initialize the system, start the sensor service, and create a Wi-Fi Access Point.
3. Using a smartphone or laptop, search for the Wi-Fi network broadcast by the device.
4. Connect to the weather station network using the device's SSID '**ByteLink**'.
5. Open a web browser and enter the IP address '**192.168.1.4**'of the **ESP32** to access the web dashboard.
6. The dashboard will display the latest temperature and humidity readings collected by the system.

System Operation.

The weather station operates automatically once powered ON. The ESP32 periodically wakes from deep sleep, reads the temperature and humidity data from the **DHT22 sensor**, updates the internal data values, and then returns to low-power mode to conserve battery energy.

Users connected to the device network can view the most recent environmental readings through the web interface.

Troubleshooting.

If the **dashboard does not load**:

- Ensure that the user device is connected to the weather station Wi-Fi network.

If sensor values are **not displayed**:

- Verify that the DHT22 sensor is correctly connected and powered.

If the system **does not power on**:

- Check that the 18650 battery is charged and properly connected.

FUTURE IMPROVEMENTS.

The current version of the weather station focuses on local environmental monitoring and web-based access through the ESP32 Access Point. Future versions of the system can expand the functionality and improve usability.

Remote Monitoring and Cloud Connectivity.

Future versions may allow users to access weather data **remotely through the internet** rather than only through a local network.

Custom Notification System.

The system could include configurable environmental thresholds that allow users to define specific temperature or humidity limits. When the measured values exceed these limits, the system could send notifications to the user through a mobile application or messaging service.

Additional Environmental Sensors.

Future versions of the weather station may integrate **additional sensors** to measure more environmental parameters such as atmospheric pressure, wind speed, or rainfall.

Solar Power Integration.

To increase system autonomy, a solar charging system could be integrated with the **18650 Li-ion** battery. This would allow the device to operate for extended periods in outdoor environments without manual battery charging.

Mobile Application.

A dedicated mobile application could be developed to provide a more convenient interface for viewing weather data, configuring system settings, and receiving alerts.